Lab 11 - Debugging Production

Uji Hipotesis, Bukan Tebak Penyebab

Baca boundary timing. Isolasi perubahan.
Verifikasi kesimpulan lewat simulasi Go.



Lukman / lukman-ss

504 adalah gejala, bukan diagnosis

README memakai request purchase orders tahunan yang lambat sebagai konteks investigasi.

SKENARIO

GET /purchase-orders?range=1y mengalami latency tinggi. Timeout terlihat oleh pengguna, tetapi belum menjelaskan penyebab.

IMPLEMENTASI

Test memanggil method handler langsung. Tidak ada HTTP server, load balancer, atau respons 504 yang dihasilkan.

TUJUAN

Belajar menguji dugaan menggunakan bukti, bukan mengubah komponen hanya karena terlihat mencurigakan.

Bedakan cerita insiden di README dari perilaku yang dimodelkan.

Petakan jalur yang benar-benar ada

Handler memanggil Service, lalu Service memanggil Repository. Semua berjalan dalam proses test.

Bagian	Pekerjaan pada simulasi
Handler	Persiapan: sleep 5 ms
Service	Sleep 10 ms, lalu FetchOrders
SlowRepository	Sleep 4 s jika 1y; selain itu 10 ms
FastRepository	Sleep 10 ms untuk semua input

DATA

Kedua repository mengembalikan satu order yang sama: ID po-1 dan Amount 100. Tidak ada query database.

Jalur pemanggilan adalah titik awal untuk memahami batas timer.

Reproduksi: input dan wiring jelas

TestReproduceBug memasang SlowRepository dan memanggil range 1y.

KONDISI

Handler dan service biasa, repository lambat, context.Background, input tahunan.

ASSERTION

Test gagal jika total kurang dari 4 detik. Artinya menerima total >= 4 detik, bukan harus lebih besar secara ketat.

```
if res.TotalDuration < 4*time.Second {
    t.Fatalf("expected request to be slow (> 4s), but took %v",
        res.TotalDuration)
}
```

Reproduksi yang spesifik memberi kondisi pembanding yang stabil.

Range besar tidak berarti data besar

Pada kode ini, rangeType hanya menentukan cabang delay. Tidak ada filtering tanggal.

Repository / input	Sleep repository	Hasil
Slow / 1y	4 detik	1 order
Slow / selain 1y	10 ms	1 order
Fast / input apa pun	10 ms	1 order

BATAS

Input invalid pun masuk cabang selain 1y. Ukuran payload tidak bertambah pada jalur tahunan.

Yang disimulasikan adalah latency, bukan biaya query atas lebih banyak baris.

Tiga hipotesis, tiga pertanyaan

README memulai investigasi dari handler, service, dan repository.

Hipotesis	Pertanyaan untuk bukti
H1: handler / JSON	Pekerjaan lokal handler lambat? JSON tidak ada di demo.
H2: service logic	Service bekerja lama atau menunggu repository?
H3: repository	Apakah perubahan repository menghilangkan delay yang dimodelkan?

Hipotesis perlu bukti yang bisa membantahnya, bukan hanya label.

Nama field bukan batas pengukuran

Empat field RequestResult tidak mewakili empat durasi independen.

Field	Cakupan aktual
HandlerDuration	Persiapan handler saja
ServiceDuration	Service + pemanggilan repository
RepositoryDuration	ServiceDuration dikurangi 10 ms
TotalDuration	Keseluruhan jalur sukses handler

PERTANYAAN KUNCI

Timer mulai dan berhenti di mana? Apakah ia mencakup pemanggilan dependency?

Baca boundary timer sebelum memakai nilai untuk menyalahkan layer.

ServiceDuration bersifat inklusif

Timer berhenti setelah FetchOrders kembali. Waktu repository masuk ke durasi service.

INTERPRETASI

Service yang terlihat lambat bisa sedang menunggu dependency. Angka tinggi tidak membuktikan business logic berat.

```
start := time.Now()
time.Sleep(10 * time.Millisecond)
orders, err := s.repo.FetchOrders(ctx, rangeType)
dur := time.Since(start)
return orders, dur, err
```

Inclusive timing mencakup pekerjaan lokal dan pemanggilan di dalamnya.

RepositoryDuration adalah turunan

Handler mengurangi konstanta 10 ms dari durasi service; tidak ada timer repository terpisah.

MEKANISME

Konstanta mengikuti sleep service yang dikonfigurasi, bukan pengukuran aktual pekerjaan lokal service.

KONSEKUENSI

Oversleep atau overhead service dapat tersisa di nilai repository. Jangan menyebutnya span independen.

```
totalDur := time.Since(totalStart)
repoDur := svcDur - 10*time.Millisecond
```

Nilai yang dihitung dari timer berbeda dari timer yang mengukur langsung.

Jangan menghitung repository dua kali

ServiceDuration sudah mengandung waktu FetchOrders. Menjumlahkan semua layer tidak menghasilkan total.

KELIRU

HandlerDuration + ServiceDuration + RepositoryDuration: waktu repository masuk dua kali.

BACA STRUKTUR

Total meliputi persiapan handler dan pemanggilan service beserta overhead. Repository berada di dalam service.

IMPLIKASI H2

ServiceDuration yang besar bukan bukti terpisah bahwa logic service adalah akar masalah.

Metrik yang ditampilkan sejajar belum tentu interval yang saling terpisah.

Hitung konfigurasi, bukan benchmark

Jumlah sleep di jalur sukses memberi gambaran desain simulasi, bukan angka runtime aktual.

Jalur	Sleep yang dikonfigurasi
SlowRepository / 1y	5 + 10 + 4000 = 4015 ms
SlowRepository / selain 1y	5 + 10 + 10 = 25 ms
FastRepository / 1y	5 + 10 + 10 = 25 ms

STATUS ANGKA

Diturunkan dari kode. Penjadwalan dan overhead memengaruhi elapsed time; tidak ada hasil eksekusi yang dilaporkan.

Selalu beri label: konfigurasi, aritmetika, atau hasil pengukuran.

Contoh log README tidak konsisten

Narasi perlu diperiksa terhadap formula yang benar-benar dijalankan.

CONTOH README

Service 12 ms, repository 4200 ms. Pasangan ini tidak sesuai repoDur = svcDur - 10 ms.

JALUR BULANAN

README menyebut 1m < 20 ms. Kode memiliki total sleep terkonfigurasi 25 ms pada jalur tersebut.

SIKAP MEMBACA

Jangan menyalin angka contoh menjadi bukti hasil test. Gunakan kode untuk menjelaskan batas interpretasi.

Dokumentasi adalah sumber konteks, bukan pengganti pemeriksaan implementasi.

Log belum menjadi assertion

TestCollectEvidence mencetak durasi sesudah memeriksa error. Ia tidak memverifikasi hubungan bottleneck.

REQUEST ID

abc123 dicetak sebagai string tetap. Tidak ada pembuatan ID unik atau tracing request di implementasi ini.

BUKTI YANG TERSEDIA

Field timing bisa diamati saat test dijalankan. Interpretasi layer lambat tidak di-assert otomatis.

```
fmt.Printf("request_id=abc123\n")
fmt.Printf("service_duration=%v\n",
    res.ServiceDuration)
```

Mencetak nilai dan memverifikasi kesimpulan adalah dua pekerjaan berbeda.

Apa yang boleh disimpulkan?

Bukti perlu mendukung mekanisme, bukan hanya nama komponen yang terlihat lambat.

Klaim	Status pada sumber
Ada delay repository 4 s	Terlihat langsung pada cabang 1y
Index database hilang	Analogi README; tidak ada database
JSON payload terlalu besar	Tidak dimodelkan; orders tidak diserialisasi
504 berhasil direproduksi	Tidak; test memanggil method langsung

Kesimpulan yang terbatas tetapi terbukti lebih berguna daripada diagnosis rekaan.

Mekanisme delay ada di cabang 1y

SlowRepository sengaja menahan jalur tahunan selama empat detik sebelum mengembalikan data.

AKAR DELAY SIMULASI

time.Sleep pada cabang input tertentu. Bukan query plan, network call, atau lock yang diukur.

```
if rangeType == "1y" {
    time.Sleep(4 * time.Second)
} else {
    time.Sleep(10 * time.Millisecond)
}
```

Sebut mekanisme konkret yang ditunjukkan sumber, bukan analoginya.

Fix: ubah satu variabel

TestVerifyFix mengganti implementasi repository, bukan merombak seluruh jalur request.

Tetap	Berubah
Input range 1y	SlowRepository menjadi FastRepository
Handler dan service	Sleep repository 4 s menjadi 10 ms
Satu order yang sama di kode	Delay yang dimodelkan

BATAS EKSPERIMEN

Ini penggantian delay buatan. Bukan demonstrasi index SQL, deployment, atau perbaikan insiden production.

Perubahan terkontrol membuat efek perubahan lebih mudah diatribusikan.

Verifikasi memakai kriteria eksplisit

TestVerifyFix gagal hanya ketika TotalDuration lebih besar dari 100 ms.

BATAS TEPAT

Durasi tepat 100 ms masih diterima. Tulisan < 100ms di pesan error tidak persis sama dengan kondisinya.

BATAS MAKNA

Threshold test lab bukan SLO production. Wall-clock timing bisa dipengaruhi kondisi eksekusi.

```
if res.TotalDuration > 100*time.Millisecond {
    t.Fatalf("expected request to be fast (< 100ms), but took %v",
        res.TotalDuration)
}
```

Pisahkan kriteria test, hasil test aktual, dan jaminan production.

Tiga test, tiga peran yang berbeda

Semua test memakai range 1y dan context.Background. Tidak semuanya membuat assertion timing.

Test	Yang diperiksa / dilakukan
TestReproduceBug	Error nil; total minimal 4 detik
TestCollectEvidence	Error nil; mencetak empat durasi
TestVerifyFix	Error nil; total maksimal 100 ms

BELUM DIPERIKSA

Payload correctness, jalur 1m, cancellation, HTTP status, dan concurrent load tidak diuji oleh tiga test ini.

Nama test tidak otomatis menjelaskan luas coverage-nya.

Error path kehilangan hasil timing

Service meneruskan error repository. Handler keluar dengan RequestResult kosong ketika ada error.

KODE TERSEDIA

Return kosong berarti caller tidak menerima timing yang sudah dikumpulkan pada jalur gagal.

COVERAGE

Kedua repository selalu mengembalikan nil error. Tidak ada repository gagal yang diinjeksi dalam test.

```
if err != nil {
    return RequestResult{}, err
}
```

Keberadaan error return bukan bukti bahwa observasi jalur gagal sudah teruji.

Context lewat, cancellation belum ada

Signature menerima context dan meneruskannya, tetapi repository tidak memeriksa pembatalan.

IMPLEMENTASI

time.Sleep berjalan tanpa select atau pemeriksaan ctx. Tidak ada deadline-aware wait dalam dua repository.

TEST

context.Background dipakai di semua test. Tidak ada context dengan deadline atau pembatalan.

KESIMPULAN

Jangan menganggap request timeout dapat menghentikan pekerjaan hanya karena ada parameter context.

Parameter API dan perilaku runtime adalah dua hal yang perlu diverifikasi.

Langkah lanjutan, bukan fitur saat ini

Setelah memahami demo, pertanyaan berikut dapat memperjelas investigasi. Ini usulan pemeriksaan.

BOUNDARY

Jika ingin durasi repository langsung, ukur di sekitar pemanggilan repository; bedakan timer inklusif dan eksklusif.

COVERAGE

Tambahkan pemeriksaan input pembanding, payload, error, dan cancellation sesuai klaim yang ingin dibuktikan.

VERIFIKASI

Jalankan test di lingkungan Go yang sesuai. Catat hasil aktual dan kondisi pengukuran, bukan hanya angka README.

Tambah bukti untuk menjawab pertanyaan tertentu, bukan sekadar menambah log.

Jalankan alur investigasinya

Perintah README dipakai dari folder lab dalam salinan repository. Root go.mod menetapkan Go 1.25.0.

STATUS MATERI

Go tidak tersedia di lingkungan penyusunan. Test dibaca, belum dijalankan; tidak ada klaim passing.

```
cd labs/11-debugging-production
go test -v -run TestReproduceBug
go test -v -run TestCollectEvidence
go test -v -run TestVerifyFix
```

Pisahkan instruksi reproduksi dari laporan bahwa reproduksi sudah dijalankan.

Uji hipotesis, batasi kesimpulan

Disiplin debugging terlihat dari hubungan antara pertanyaan, bukti, dan perubahan.

MENTAL MODEL

Debugging is testing hypotheses, not guessing causes.

ALUR

Reproduksi spesifik → hipotesis → boundary timing → bukti → satu perubahan → verifikasi.

KESIMPULAN LAB

Delay buatan di repository menjelaskan jalur lambat yang dimodelkan. Penyebab database production belum dibuktikan.

Bukti menentukan seberapa jauh sebuah klaim boleh dibuat.

Gejala memulai. Bukti menentukan.

Software Engineering Lab
Lab 11 - Debugging Production

```
README.md
main.go
main_test.go
```

REPOSITORY

github.com/lukman-ss/software-engineering-lab
/tree/main/labs/11-debugging-production

Lukman / lukman-ss

Sumber: commit 82af988b89439fa92b1200406c5afbd59ce72a56
Test dibaca, tidak dijalankan.
Jumlah sleep adalah konfigurasi, bukan hasil benchmark.